Object Oriented Programming [FECEg-3142]

CHAPTER THREE:

Classes and Objects- a deeper look

Outline

- Classes, objects
- Methods
- Constructors
- Composition
- The Static and final key words
- Constructors and destructors
- Using the UML class and Composition diagrams

Introduction to classes and objects

- Object-oriented programming (OOP) involves programming using **objects**.
- An **object** represents an **entity** in the real world that can be distinctly identified.

  For example, a **student**, a **desk**, a **circle**, a **button**, and even a loan can all be viewed as objects.
- An **object** has a unique identity, **state**, and **behavior**.

Cont'd...

- The **state** of an object (also known as its **properties** or **attributes**) is represented by **data fields** with their current values.
  - A **circle object**, for example, has a data field **radius** , which is the property that characterizes a circle.
  - A **rectangle object** has data fields **width** and **height** , which are the properties that characterize a rectangle.

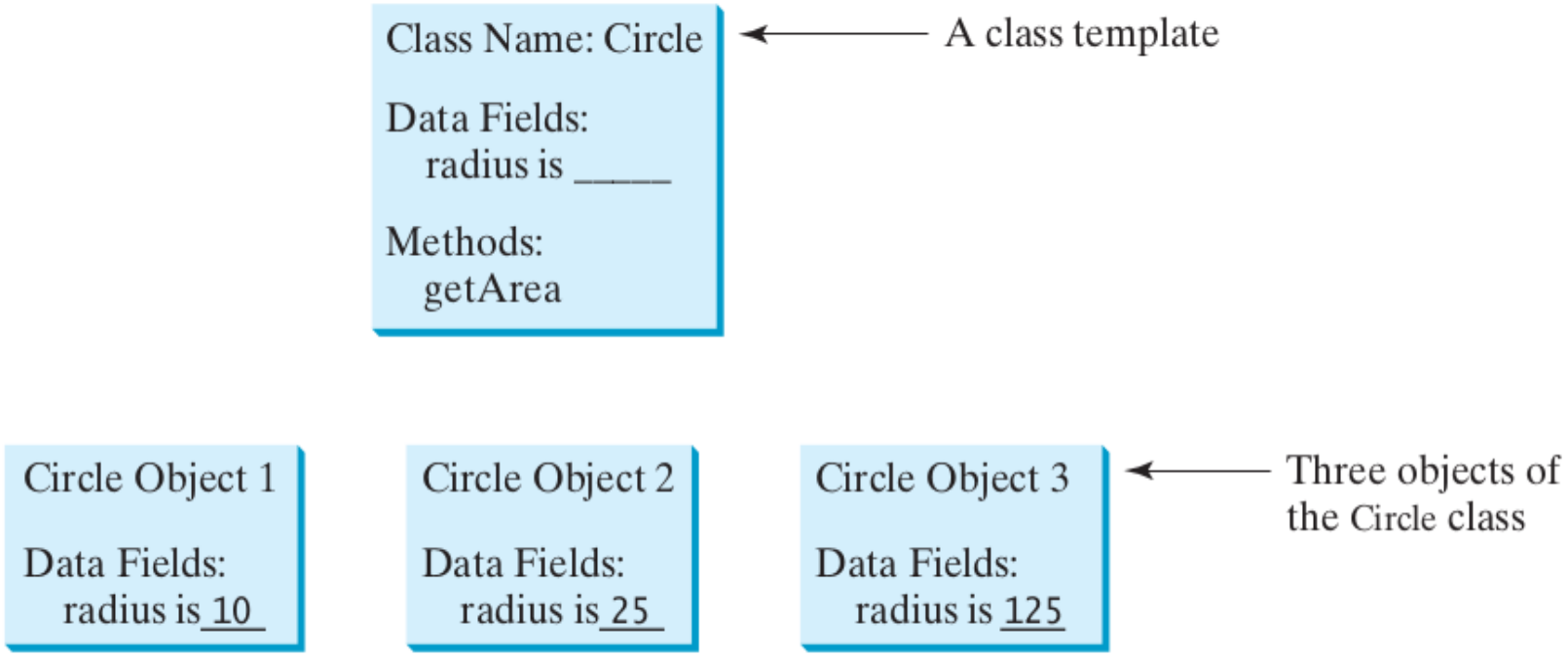
Cont'd...

- The **behavior** of an object (also known as its **actions**) is defined by **methods**.
  - To invoke a method on an object is to ask the object to perform an action.
  - For example, you may define a method named **getArea()** for circle objects.
  - A circle object may invoke **getArea()** to return its area.

Cont'd...

- **Objects** of the same type are defined using a common **class**.
- A **class** is a template, **blueprint**, or **contract** that defines what an object's **data fields** and **methods** will be.
- An **object** is an instance of a **class**.
- You can create many instances of a class.
- Creating an instance is referred to as **instantiation**.
- The terms **object** and **instance** are often interchangeable.

Cont'd...



A class is a template for creating objects.

Example

```
public class TestCircle1 {
/** Main method */
public static void main(String[] args) {
// Create a circle with radius 1.0
Circle1 circle1 = new Circle1();
System.out.println("The area of the circle of radius "
+ circle1.radius + " is " + circle1.getArea() );
// Create a circle with radius 25
Circle1 circle2 = new Circle1(25);
System.out.println("The area of the circle of radius "
+ circle2.radius + " is " + circle2.getArea());
// Create a circle with radius 125
Circle1 circle3 = new Circle1(125);
System.out.println("The area of the circle of radius "
+ circle3.radius + " is " + circle3.getArea());
// Modify circle radius
circle2.radius = 100;
System.out.println("The area of the circle of radius "
+ circle2.radius + " is " + circle2.getArea() );
}
}
```

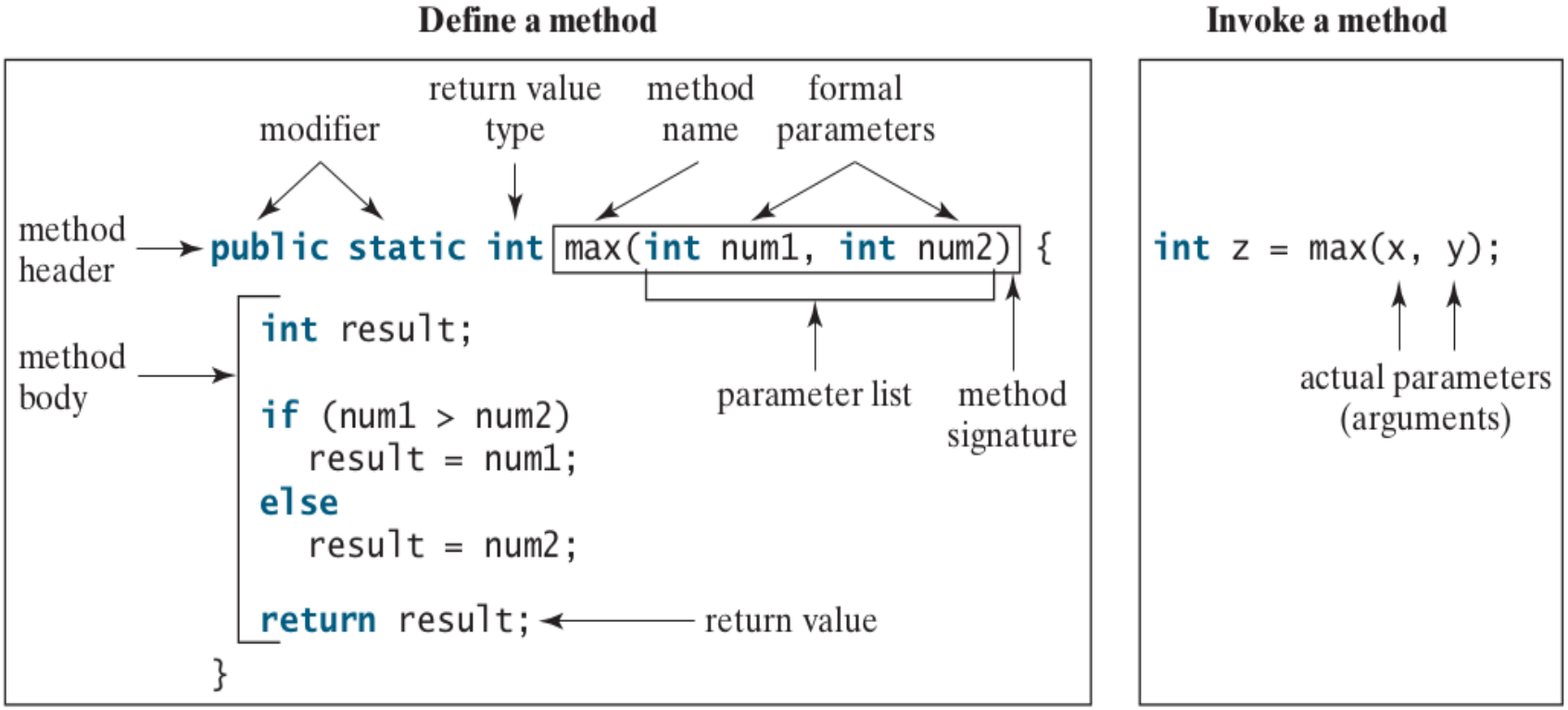
```
public class Circle1 {
  double radius ;
/** Construct a circle with radius 1 */
  Circle1() {
    radius = 1.0;
  }
/** Construct a circle with a specified
radius*/
  Circle1(double newRadius) {
    radius = newRadius;
  }
/** Return the area of this circle */
  double getArea() {
    return radius * radius * Math.PI;
  }
}
```

Methods

- A **method** is a collection of statements grouped together to perform an operation.
- Methods can be used to reduce redundant code and enable code reuse.
- Methods can also be used to modularize code and improve the quality of the program.
- The **syntax** for defining a **method** is as follows:

```
modifier returnValueType methodName (list of parameters) {
    // Method body;
}
```

Cont'd...



A method definition consists of a method header and a method body.

Cont'd...

- If a **method** returns a **value**, it is called a **value-returning** method, otherwise it is a **void** method.
- The variables defined in the method header are known as formal parameters or simply parameters.
- A parameter is like a placeholder.
- When a method is invoked, you pass a value to the parameter. This value is referred to as an actual parameter or argument.
- The parameter list refers to the type, order, and number of the parameters of a method.
- The method name and the parameter list together constitute the method signature.

Calling a Method

- In creating a method, you define what the method is to do.
- To use a method, you have to call or invoke it.
- There are two ways to call a method, depending on whether the method returns a value or not.

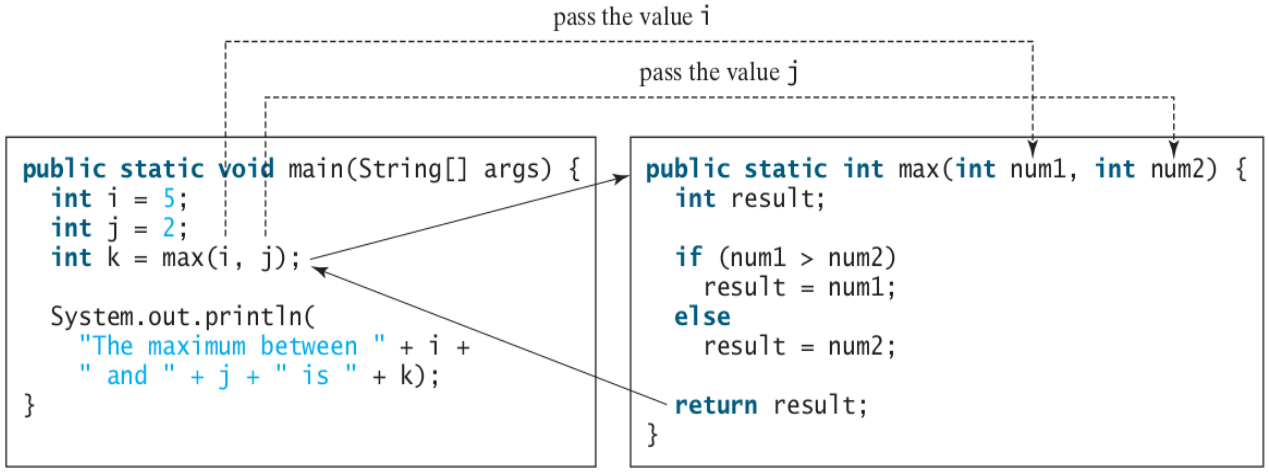
```
int larger = max(3, 4);

System.out.println(max(3, 4));
```

Cont'd...

```
public class TestMax {
   /** Main method */
      public static void main(String[] args) {
         int i = 5;
         int j = 2;
         int k = max(i, j) ;
         System.out.println("The maximum between " + i + " and " + j + " is " + k);
      }
      /** Return the max between two numbers */
      public static int max(int num1, int num2) {
         int result;
         if (num1 > num2)
            result = num1;
         else
            result = num2;
         return result;
      }
}
```

Cont'd...



- When the max method is invoked, the flow of control transfers to it.
- Once the max method is finished, it returns control back to the caller.
- When calling a method, you need to provide arguments, which must be given in the same order as their respective parameters in the method signature.

Constructors and destructors

Constructors

- **Constructors** are a special kind of method.
- They have three peculiarities:
  - A constructor must have the **same name** as the class itself.
  - Constructors do not have a **return type**—**not even void** .
  - Constructors are invoked using the **new** operator when an object is created.
  - Constructors play the role of **initializing** objects.

Cont'd...

- Like regular methods, constructors can be overloaded, making it easy to **construct objects** with **different initial data values**.
- **Constructors** are used to **construct objects**.
- To construct an object from a class, invoke a constructor of the class using the new operator, as follows:

```
new ClassName ( arguments ) ;
```

Cont'd...

- A class may be defined without constructors.
  - In this case, a no-arg constructor with an empty body is implicitly defined in the class.
  - This constructor, called a default constructor, is provided automatically only if no constructors are explicitly defined in the class.
- Objects are accessed via object **reference variables**, which contain references to the objects.
- Such variables are declared using the following syntax:

  ClassName objectRefVar;

Cont'd...

- A class is a reference type, which means that a variable of the class type can reference an instance of the class.
- The following statement declares the variable myCircle to be of the Circle type:

```
Circle myCircle;
```

- The variable myCircle can reference a Circle object. The next statement creates an object and assigns its reference to myCircle :

```
myCircle = new Circle();
```

Cont'd...

- After an object is created, its **data can be accessed** and its **methods invoked** using the **dot operator** (**.**), also known as the **object member access operator**:
  - objectRefVar.dataField dot operator references a data field in the object.
  - objectRefVar.method(arguments) invokes a method on the object.

Destructors

- The constructor—is called automatically when an object is first created.
- Another function is called automatically when an object is destroyed.
  - Such a function is called a destructor.
- A destructor has the same name as the constructor (which is the same as the class name).
- The most common use of destructors is to deallocate memory that was allocated for the object by the constructor.

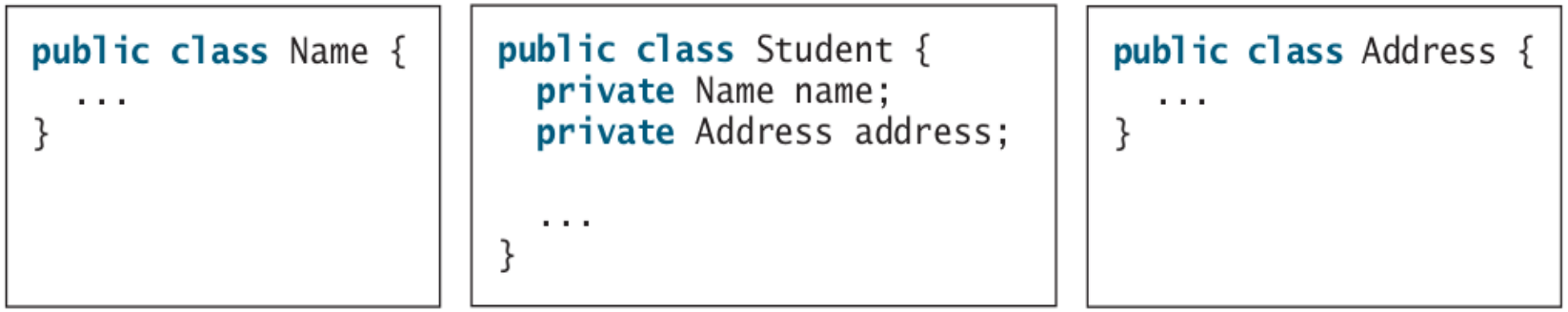
Composition

- An object can contain another object, the relationship between the two is called **composition**.
- Composition is actually a special case of the aggregation relationship.
- Aggregation models has-a relationships and represents an ownership relationship between two objects.
- The owner object is called an aggregating object and its class an aggregating class.
- The subject object is called an aggregated object and its class an aggregated class.

Cont'd...

- An object may be owned by several other aggregating objects.
  - If an object is exclusively owned by an aggregating object, the relationship between them is referred to as **composition**.
- An aggregation relationship is usually represented as a data field in the aggregating class.

  Example



Modifiers

- **Modifiers** are keywords that you add to those definitions to change their meanings.
- The Java language has a wide variety of modifiers, including the following:
  - Modifiers for controlling access to a **class**, **method**, or **variable**: **public**, **protected**, and **private**.
  - The **static** modifier for creating **class methods** and **variables**
  - The **final** modifier for finalizing the implementations of classes, methods, and variables
  - The **abstract** modifier for creating abstract classes and methods
  - To **use a modifier**, you include its **keyword** in the definition of a class, method, or variable.

Access Control for Methods and Variables

- The modifiers that you will use the most often control access to **methods** and **variables**:
  - Public
  - private
  - protected
- These modifiers determine which **variables** and **methods** of a class are **visible** to other classes.
- By using access control, we can dictate how our class is used by other classes.
- Some variables and methods in a class are of use only within the class itself and should be hidden from other classes. This process is called **encapsulation**:

Cont'd...

- **Encapsulation** is the process that prevents class variables from being read or modified by other classes.
- The only way to use these variables is by calling methods of the class, if they are available.
- The Java language provides four levels of access control: public , private , protected , and a default level specified by using none of these access control modifiers.

Cont'd...

- **Default Access**: A variable or method declared without any access control modifier is available to any other class in the same package.

Private Access:

- To completely hide a method or variable from being used by any other classes, use the private modifier.
  - The only place these methods or variables can be accessed is from within their own class.
- Private variables are useful in two circumstances:
  - When other classes have no reason to use that variable
  - When another class could wreak havoc by changing the variable in an inappropriate way

Cont'd...

Public Access

- The public modifier makes a method or variable completely available to all classes.

Protected Access

- The third level of access control is to limit a method and variable to use by the following two groups:
  - Subclasses of a class
  - Other classes in the same package

The Static and final key words

Static Variables and Methods

Static Methods

- A **static method** is one that can be used without a calling object. With a static method, you normally use the class name in place of a calling object.
- When you define a static method, you place the keyword **static** in the heading of the definition.

```
public static int maximum( int n1, int n2)
{
    if (n1 > n2)
        return n1;
    else
        return n2;
}
```

Cont'd...

- So if the above definition of the method maximum were in a class named SomeClass , then the following is a sample invocation of maximum :

```
int budget = SomeClass.maximum(yourMoney, myMoney);
```

Cont'd...

Static Variables

- A static variable belongs to the class as a whole. All objects of the class can read and change the static variable. Static variables should normally be private, unless they happen to be defined constants.
- **SYNTAX**
  - private static Type Variable_Name;
  - private static Type Variable_Name = Initial_Value;
  - public static final Type Variable_Name = Constant_Value;

EXAMPLES

- private static String lastUser;
- private static int turn = 0;
- public static final double PI = 3.14159;

Final Classes, Methods, and Variables

- The final modifier is used with classes, methods, and variables to indicate that they will not be changed.
- It has different meanings for each thing that can be made final, as follows:
  - A final class cannot be subclassed.
  - A final method cannot be overridden by any subclasses.
  - A final variable cannot change in value.

Cont'd...

Variables

- Final variables are often called constant variables (or just constants) because they do not change in value at any time.
- With variables, the final modifier often is used with static to make the constant a class variable.
- If the value never changes, you don't have much reason to give each object in the same class its own copy of the value.
- They all can use the class variable with the same functionality.
- The following statements are examples of declaring constants:

```
public static final int TOUCHDOWN = 6;
static final String TITLE = "Captain";
```

Cont'd...

Methods

- Final methods are those that can never be overridden by a subclass.
- You declare them using the final modifier in the class declaration, as in the following example:

```
public final void getSignature() {
// body of method
}
```

Cont'd...

Classes

- You finalize classes by using the final modifier in the declaration for the class, as in the following:

```
public final class ChatServer {
// body of method
}
```

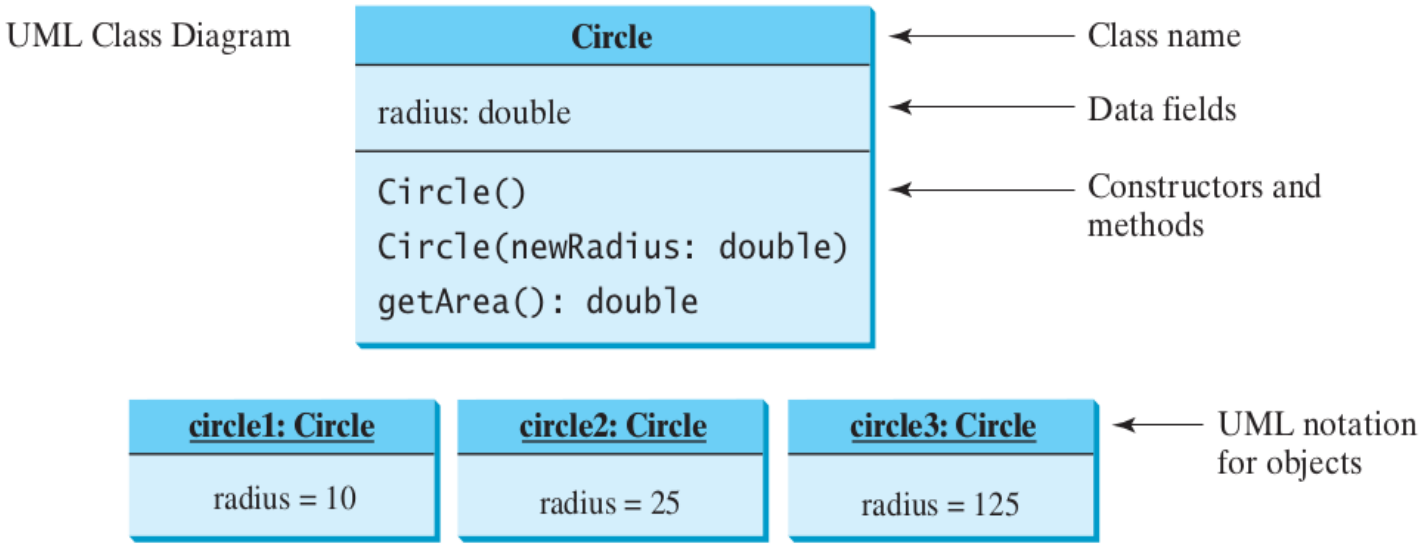
- A final class cannot be subclassed by another class.
- As with final methods, this process introduces some speed benefits to the Java language at the expense of flexibility.

Using the UML class and Composition diagrams

UML class diagram

- **Class diagram** are used when developing an object-oriented system model to show the classes in a system and the associations between these classes.
- Each node in a class diagram is labeled with its **class name**.
- The class node may also contain lists of **data attributes** and **method prototypes**.
- The visibility of attributes or methods can be indicated by prefixing their names with a (**public**) or (**private**).
- An **association** line indicates that there is a linkage between two classes.

Cont'd...



- In the class diagram, the data field is denoted as

  dataFieldName: dataFieldType

- The constructor is denoted as

  ClassName(parameterName: parameterType)

- The method is denoted as

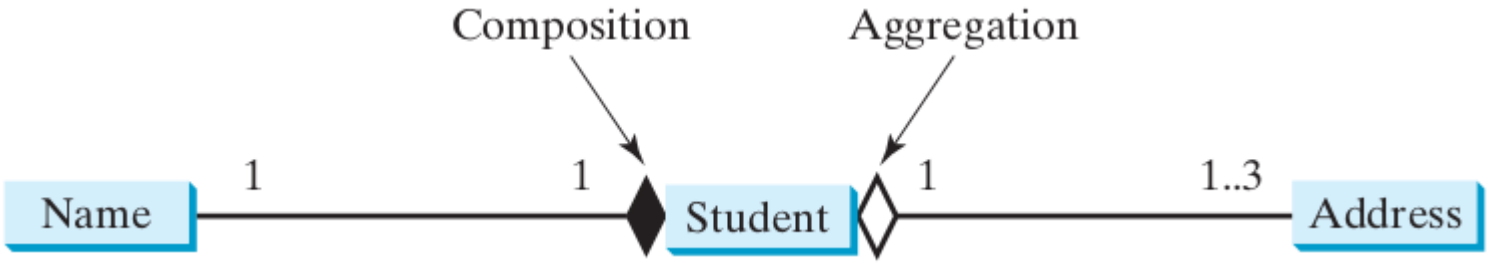
  methodName(parameterName: parameterType): returnType

Cont'd...

Composition diagrams

- In UML, a **filled diamond** is attached to an **aggregating class** to denote the **composition relationship** with an **aggregated class**, and an **empty diamond** is attached to an **aggregating class** to denote the **aggregation relationship** with an **aggregated class**.

  Example



  - A student has a name and an address.

End